

Viral Protein Language Model Workflow

The viral protein language model, as specified on the left-hand-side of Fig 3, has the following workflow:

1 Entry Point

Each type of virus (namely `flu`, `cov` and `hiv`) has its own, mainly identical, entry files, with a list of command-line arguments (explained in Section 2 below).

The training pipeline is setup by:

- Setting up a vocabulary list, which maps each amino acid to an index which starts at 1
- Calls `setup()`, which:
 - Loads and processes data by applying a filter and writing metadata.
 - Obtains the untrained model, as specified `language_model.py`

This represents the initial “Input context” layer in Fig 3, and equations (1) and (2).

2 Processing arguments

After setting up, the program handles the following command line arguments as follows.

- `checkpoint (str)` – if not none:

The previous weights stored in the specified cached file are the loaded to the model, in order to continue on previous training results

- `train (bool)` – if True:

Calls `mutation.batch_train()` to train the model. To control number of epochs, one should modify `model.n_epochs_` in that function. The function:

- Splits the sequences into batches of 5000
- For each batch, run training through `fit_model()` (delegated through `train_test()`):
 - Featurizes sequences by converting a dictionary of sequences into arrays used by the models
 - Runs the `fit()` method on the model instance (explained in detail in Section 3 Model Training in Detail).

This represents the “BiLSTM layers” in Fig 3.

- `train_split (bool)` or `test (bool)` – if True:

Calls `train_test()` directly:

- If `train_split` is True, one fitting process is run for the `train_seqs` portion of the data, split by the `split_seqs()` function defined in each caller file.
- If `test` is True, reports cross-entropy performance through `report_performance()`

- `embed (bool)` – if True:

Calls `analyze_embedding()` to generate sequence embeddings and visualizes clusters. This acts as the “semantic embedding” layer in Fig 3.

- `semantics (bool)` – if True:

- Loads data from relevant data sets
 - For each dataset, calls `mutation.analyze_semantics()` to compute and cache semantic change in accordance with Equation 3, as well as grammaticality in accordance with Equation 4.
 - `cached_semantics.cached_escape()` immediately loads its output, generates escape prediction maps (Figs 4-6) and discovery curves. It also prints out escape detection performance of the model

This represents the final “output grammaticality” layer in Fig 3.

- `combit (bool)` – if True:

- Loads data from relevant data set(s)
- For each dataset, calls `mutation.analyze_comb_fitness()` to generate combinatorial fitness scatter plots

- `reinfection (bool)` – if True:

- Loads data from relevant data set(s)
- For each dataset, calls `mutation.analyze_reinfection()` to compute and cache semantic scores for reinfection scenarios
- Visualizes reinfection based on the cached semantic scores

3 Models in detail

The base `LanguageModel` class has the following key methods, used by all subtypes:

`model.fit()`:

- Calls `split_and_pad()` on each subclass, which splits data into a 2D-list (or equivalent data structure) of `[X, Y]`, where `X` can be seen as `[X_Pre, X_Post]`, with `X_Pre`, `X_Post` having the same length of `X` and padded with 0s, containing sequences before and after specified position `i`, and the `Y` contains the ground truth tokens at each position.
- Sets up the optimizer and the checkpoint.
- Calls `fit()` in the Keras instance of each model

`model.predict()`:

- Calls the Keras instance to make a prediction based on the `X` given by `split_and_pad()`

The `BiLSTMLanguageModel`, in particular, was used for the paper. As specified in equation (5), a softmax function is placed in the final layer.

4 Use in Further Data Analysis

The 3D spike protein escape potential maps, as shown in Fig 11, are generated from the PyMol commands produced by `color_protein.py`, from the escape and region files previously cached.